

实验5 CAIL2020 多跳法律问答系统 (Baseline)

从理论到案例的具体结构，流程和主要函数和关键代码做了理解性的注释和梳理，目前对数据处理这块能够做分析，但是需要标准才能很好的运行文件，基础还很薄弱，以后会加强这方面的训练。此次报告内容很长，用主要标题和下面的七个大点做分层，方便定位内容。

项目目标

本项目是，一个多跳问答（Multi-hop QA / MRC）任务，给定：

- 一个法律相关上下文（context）
- 一个提问（question）

要求模型返回：

- 答案（answer）：是或否
- 支持证据（supporting facts）：从上下文中抽取的关键句

对

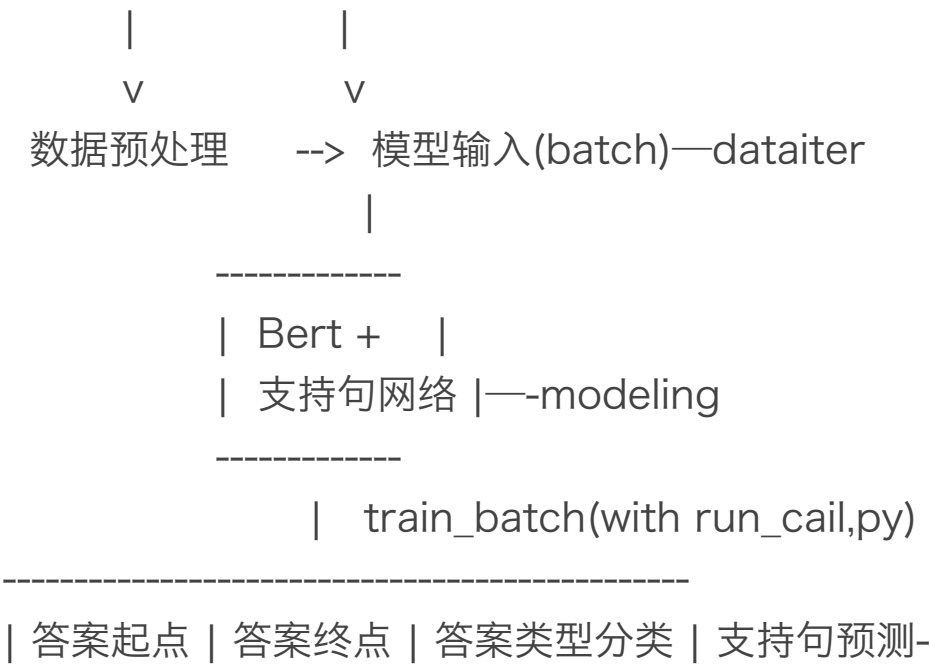
主要文档结构说明

代码源（文件名）	功能描述	备注说明
命令脚本preprocess.sh	集中可变参数，一键调取参数和命令，创建环境，完成数据预处理脚本data——precess，从原始 JSON 文件生成训练所需的中间数据文件，后不再单独解释sh	输入人工标注的 train.json 和 dev.json，mkdir precess文件夹，，输出和保持四个gz文件；这里bert模型bertokenizer
dataiter.py	Data送入模型的主要辅助工具，Iter在逻辑上实现把数据变成张量送入模型，helper连接器，让数据流转更稳定，简单	输入tokenid列表输出对齐的张量，用到mapping和mask，做好数据映射，预测文档可以溯源
data_process.py	主要数据处理函数，读取 JSON，调用分词器生成特征文件	用哈工大的RoBERTa 分词器–本质是chinesebert，所以和 berttokenizer兼容
run_cail.py, train.sh	模型训练主程序，包含训练、验证及预测流程，这里添加了step级的loss图供前期调试学习可视化加深理解，输出png图	支持多参数配置，调用config，训练和评估入口，读取proceed四个文件创建output文件，存储epoch级别checkpoints和json预测文件，最后一个json文件用于功能评估
model/GFN.py	模型定义文件，基于 RoBERTa 的多跳问答模型实现，主要用到modeeling.py,核心模型结构，包含 start/end 预测 span,ysno预测分类	GFN.py不完整，是竞赛用模型，兼容更多， 本次项目用不到不调用
evaluate.py/evaluate.sh	预测结果的评测脚本，计算f1,em等分类和支持句的准确率等指标	输入预测文件 pred_seed_13_epoch_10_99999.json和测试集标注文件 dev_goldtruth.json
pred_seed_13_epoch_10_99999.json	模型输出，包含 answer 和 supporting facts 的格式示例	在output submission的train_v1下

Config.py	本案例数据处理，训练和评估所有参数的定义，供运行文件调用，模型微调的入口，可以调整batch大小，学习率，adamw等训练策略	
Tools文件夹其他工具文件	datahelper, utility, 封装函数装饰器，时间模型计算公式、log等函数方便主程序调用	

项目结构与模型流程图

数据(json) > Example对象(readexample) --> InputFeatures (def converttofeature)



predict ()

核心代码注释与说明

读取数据集json

Json结构如下[{'_id': 130,
 'context': [['本院根据以上有效证据及当事人质证意见认定以下事实：
2013年9月18日下午17时15分左右原告王相青在工作时间， '
 [' '因工作原因骑电动车前去安阳市汽车站接林州客户网点捎来的保
单， '
 '因躲避行人而侧翻， '
 ' '王相青支付劳动能力鉴定费300元。']],
 'question': '被告王相青是否没有收到应享有的工伤待遇？ ',
 'answer': 'yes',
 'supporting_facts': [[], ['本院根据以上有效证据及当事人质证意见认
定以下事实： 2013年9月18日下午17时15分左右原告王相青在工作
时间， ', 12]]],

一、数据处理模块 (data_process.py)

Class Example —数据拼接成bert模型需要的张量 [CLS] Q [SEP] token, token2 ... [SEP],提取所有关键变量组件问题-答案支持, 句子和查句子的位置变量

```
class Example(object):
```

```
    def __init__(self, ...):
```

```
        self.qas_id = qas_id # 问题唯一标识
```

```
        self.qas_type = qas_type # 问题类型
```

```
        self.doc_tokens = doc_tokens # 上下文tokens
```

```
        self.question_text = question_text # 原始问题文本
```

```
        self.sent_names = sent_names # 每个句子的(title, index)标
```

识

```
        self.sup_fact_id = sup_fact_id # 支持句在句子列表中的索引
```

```
        self.start_position = start_position # 答案起点token位置
```

```
        self.end_position = end_position # 答案终点token位置
```

```
def read_examples:
```

本任务最核心的部分之一。处理json复制给上面的init变量, 返回单个标准样本参与模型训练。

```
判断分类功能: JUDGE_FLAG =  
str(orig_answer_text).lower() in ['yes', 'no', 'unknown']  
ans_start_position.append(char_to_word_offset[start_char_pos  
ition])  
#  
ans_end_position.append(char_to_word_offset[end_char_positi  
on])
```

判断支持句在哪: if local_sent_name in sup_facts:
然后写成预测:

```
sent_end_word_id = len(doc_tokens) - 1  
  
sent_start_end_position.append((sent_start_word_id,  
sent_end_word_id))
```

```
# sup_facts_sent_id.append(sent_id)  
char_to_word_offset = []  
# FIND_FLAG = False
```

Def convert_examples_to_features 函数: token进一步变成
token_id,一个样本变成一个feature列表

```
# 将Example对象转换为InputFeatures, 生成模型所需  
输入格式  
# 包括输入ids、segment ids、mask、句子span索引等
```

二、迭代器模块 (data_iterator_pack.py中的 iter 方法)

iter—doctokens长度 512，给所有位置和id一个模型看得懂的序列号,label编号 span= 【y1,y2】，用于标记答案的位置和长度

```
def __iter__(self):
```

```
    # 构造一个batch，包含所有模型输入字段
```

```
    context_idx = torch.LongTensor(self.bsz, 512) # 长度固定为512，用于BERT编码器输入token
```

```
    context_mask = torch.LongTensor(self.bsz, 512) # 对应的attention mask，1表示有效token，0表示padding，确保模型只关注有效部分
```

```
    segment_idx = torch.LongTensor(self.bsz, 512) # 用于区分query和context
```

```
    query_mapping = torch.Tensor(self.bsz, 512).cuda(self.device) # 标记输入序列中哪些token属于查询部分(query)，一般为one-hot
```

```
    start_mapping = torch.Tensor(self.bsz, self.sent_limit, 512).cuda(self.device) # 标识每个句子在输入token序列中的起始位置，是句子级别的one-hot编码，用于支持句预测
```

```
    all_mapping = torch.Tensor(self.bsz, 512, self.sent_limit).cuda(self.device) # 实现token到句子的映射，辅助模型进行多粒度信息交互
```

```

    y1 = torch.LongTensor(self.bsz).cuda(self.device) # 答案起
点标签学习答案的具体位置
    y2 = torch.LongTensor(self.bsz).cuda(self.device) # 答案终
点标签
    q_type = torch.LongTensor(self.bsz).cuda(self.device) # 答
案类型标签，这里是“yes/no/unknow和else”多类别分类信息
    is_support = torch.FloatTensor(self.bsz,
self.sent_limit).cuda(self.device) # 支持句标签，浮点型矩阵，指
示每个句子是否为支持句

    while True:
        # 遍历每个样本，构建batch
        # 并将输入组织成张量形式返回模型使用
        for j, sent_span in
enumerate(case.sent_spans[:self.sent_limit]):
            is_sp_flag = j in case.sup_fact_ids    #支持句标
签，用于判断，这里就是最后一个j代表token长度
            start, end = sent_span
            if start < end:
                is_support[i, j] = int(is_sp_flag) #截取句子
span
                all_mapping[i, start:end+1, j] = 1    #是span
的句子token标记为1，否则0，和sansentence一起输出sp，和logit

```

```
start_mapping[i, j, start] = 1    #是answer  
start的token标记为1， 否则0， 结合end预测答案的起止logits  
yield { ... }
```

三、模型结构 (modeling.py)

SimplePredictionLayer

```
class SimplePredictionLayer(nn.Module):  
    def __init__(self, config):  
        self.sp_linear = nn.Linear(self.input_dim, 1) # 支持句预测  
        self.start_linear = nn.Linear(self.input_dim, 1) # 答案起点  
        self.end_linear = nn.Linear(self.input_dim, 1) # 答案终点预  
        self.type_linear = nn.Linear(self.input_dim,  
config.label_type_num) # 答案类型分类
```

```
    def forward(self, batch, input_state):  
        # 使用context表示预测起点、终点、类型、支持句  
        return start_logits, end_logits, type_logits, sp_logits,  
start_pos, end_pos
```

BertSupportNet

```
class BertSupportNet(nn.Module):  
    def __init__(self, config, encoder):  
        self.encoder = encoder # BERT/RoBERTa/Lawformer的
```

tokenizer模型嵌入

```
self.graph_fusion_net = SupportNet(config) # 支持句融合  
预测结构
```

```
def forward(self, batch):
```

```
    # 输入通过BERT编码后，交给SupportNet做任务预测，是训练  
    过程中主要的工作，让bersupornet学习分类和span句子在人工标注  
    的特征，生成logit, loss，通过backward输出更好的权重后在训练时  
    存入checkpoint，输出预测
```

```
    return self.graph_fusion_net(batch)
```

这里要考虑batch的大小，因为已经是512个token，如果batch过大，对于GPU不足会造成运行困难，时间过长，不能运行等问题，所以要合理根据环境设置batch大小，此处为默认的32，就是模型一次处理32个样本case，并通过学习率warmup和step-acumulation等工具减少运算压力。

四、训练模块 (run_cail.py)

run_cail.py

- └── train_epoch()

- └── train_batch(model, batch)

- └── start_logits, end_logits, type_logits, sp_logits = model(batch)

- └── compute_loss(...) ← 4类输出都参与

- └── loss.backward(), optimizer.step()

compute_loss

```
def compute_loss(batch, start_logits, end_logits, type_logits, sp_logits, ...):
```

```
    # 计算三类损失函数: span loss + 类型 loss + 支持句 loss
```

```
    return loss, loss1, loss2, loss3
```

本项目返回数值的只有loss2,3，模型本身设置了5个loss，这里由于模型要支持更多的任务运算，而项目是一对一问答分类+span，所以在仅在分类和span预测上有数值这里的loss0不等于compute的loss2+loss3

predict

```
@torch.no_grad()
```

```
def predict(...):
```

```
    # 在验证集上评估当前模型，生成预测json文件，保存支持句和答案
```

关键代码: new_answer_dict, sp_dict.update({cur_id: cur_sp_pred}),

prediction = {'answer': new_answer_dict, 'sp': sp_dict}—拼接训练好的context句子和支持句子：文件附件：outputsubmission里面每个epoch输出都可以作为训练的功能评估，一般取最新的输出文件做整体评估

```
with open(prediction_file, 'w', encoding='utf8') as f: #生成预测答案的文档用于评估
```

```
    json.dump(prediction, f, indent=4, ensure_ascii=False):
```

可视化loss效果，后加载loss绘图提取此处数据

```
#  每步记录 loss2, 3
```

```
    step_losses2.append(loss_list[2].item())
```

```
    step_losses3.append(loss_list[3].item())
```

```
    test_loss_record.append(sum(total_test_loss[:3]) /  
len(dataloader)) , 因为频繁在epoch1做数据验证，所以后改为用  
step做记录，运行良好后改epoch 10, loss可视化就没有什么效果了  
train_batch-训练的核心函数
```

```
def train_batch(model, batch):
```

```
    # 承接模型输出的数据，模型前向传播、反向传播和优化器更新；  
项目里承接模型训练的数据流logit环节，完成batch级别的计算loss
```

```
    start_logits, end_logits, type_logits, sp_logits,  
start_position, end_position = model(batch)
```

```
    loss_list = compute_loss(batch, start_logits, end_logits,  
type_logits, sp_logits, start_position, end_position)
```

```
    batch = next(iter(data_loader))
```

```
batch['context_mask'] = batch['context_mask'].float()
train_batch(model, batch) 送数据进去后按照batch训练
```

这里要说明 分类和span分别是iloss2和3，计算loss函数可以改为loss=loss2+loss3做调试，这个尝试没来得及做。

项目有几个地方函数没有return，如果在这里用global global_step, total_train_loss, step_losses，会加大不熟悉代码寻找数据流的困难

在batch里面设置梯度累积，

```
args.gradient_accumulation_steps > 1
```

```
def train_epoch(...):
```

 # 训练一轮模型需要的结果输出，保存等协助功能，同时可以每隔一定步数进行中间预测评估

关键代码：

```
    predict(model, eval_dataset, dev_example_dict,
dev_feature_dict,
            join(args.prediction_path, 'pred_seed_{}_epoch_{}
_99999.json'.format(args.seed, epc)))
    model_to_save = model.module if hasattr(model, 'module')
```

五、参数模块（Config.py）

模型配置由 set_config() 管理，自动保存：output/checkpoints/train_v1/run_settings.json重要参数如下：

参数名	含义	示例值
--bert_model	使用的编码器路径（如 lawformer）	baseline/lawformer
--input_dim	BERT 输出维度	768
--label_type_num	答案类型分类数	4（yes/no/unknown/span）
--batch_size	训练时的 batch size	2
--epochs	总训练轮数	10
--gradient_accumulation_steps	梯度累计步数	1
--type_lambda	答案类型 loss 权重	1
--sp_lambda	支持句 loss 权重	5
--sp_threshold	支持句预测阈值（用于推理）	0.5
--trained_weight	是否从某个 epoch checkpoint 开始	ckpt_seed_68_epoch_6_910.pth

运行和优化代码

六、代码修改内容

1. 本地化训练配置（Preprocess.sh & Train.sh）

Preprocess.sh

```
source /root/autodl-tmp/bertm/torch26_env/bin/activate
INPUT_TRAIN_FILE=../data/train.json
INPUT_DEV_FILE=../data/test.json
OUTPUT_DIR=../data/processed # 注意：该路径应与 train.sh 中的 data_dir 保持一致
tokenizer_path='autodl-tmp/bertm/预训练模型应用实例-更新版/chinese-roberta-wwm-ext'
```

Train.sh

```
source /root/autodl-tmp/bertm/torch26_env/bin/activate
bert_dir='/root/autodl-tmp/bertm/预训练模型应用实例-更新版/chinese-roberta-wwm-ext'
```

训练日志（示例：Epoch 10）

```
train_v1 -- In Epoch10:
Avg-LOSS0/batch/step: nan
Avg-LOSS1/batch/step: nan
Avg-LOSS2/batch/step: 0.1225
Avg-LOSS3/batch/step: 0.7817
Avg-LOSS4/batch/step: 0.0
```

Test Loss0: nan
Test Loss1: nan
Test Loss2: 0.3849
Test Loss3: 2.1676
Test Loss4: 0.0

2. 可视化模块修改 (loss 记录 + 曲线绘图)

在 `runcaill.py` 中的 `train_epoch()` 和主函数中新增:

```
# 每步记录 loss2、loss3
step_losses2.append(loss_list[2].item())
step_losses3.append(loss_list[3].item())
```

新增可视化函数:

```
def plot_metrics(losses, save_path="output/
metrics_summary.png"):
    epochs = list(range(1, len(losses) + 1))
    plt.figure(figsize=(10, 6))
```

```
# Loss 曲线
plt.plot(epochs, losses, marker='o', label="Test Loss")

plt.title("Training Summary: Test Loss")
plt.xlabel("Epoch")
plt.ylabel("Metric Value")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(save_path)
```

输出图示例：

metrics_steps_20250728_184749.png

3. 加入新模型 Lawformer

修改 Train.sh / Preprocess.sh 中 bert_dir 和 tokenizer_path:

```
bert_dir='/root/autodl-tmp/bertm/Lawformer'
```

```
tokenizer_path='/root/autodl-tmp/bertm/Lawformer'
```

runcail.py 中 encoder 替换为 AutoModel 加载 Lawformer:

```
from transformers import AutoModel, AutoConfig
```

```
roberta_config =
```

```
AutoConfig.from_pretrained(args.bert_model)
```

```
encoder = AutoModel.from_pretrained(args.bert_model)
```

procpy.py 中 tokenizer 替换为 AutoTokenizer:

```
from transformers import AutoTokenizer
```

```
tokenizer =
```

```
AutoTokenizer.from_pretrained(args.tokenizer_path)
```

4. 引入外部数据 (CAIL 数据) 与调试尝试记录

Token粒度评估结果 (CAIL 数据, epoch 6) :

```
Evaluating prediction file: output/submissions/train_v1/  
pred_seed_24_epoch_6_99999.json
```

```
Using gold file: ../data/dev.json
```

```
{
  "em": 0.033377308707124013,
  "f1": 0.07277722831663301,
  "prec": 0.11018657381536649,
  "recall": 0.06488013333177498,
  "sp_em": 0.6286939313984169,
  "sp_f1": 0.569986807387863,
  "sp_prec": 0.5684696569920844,
  "sp_recall": 0.5730211081794195,
  "joint_em": 0.03205804749340369,
  "joint_f1": 0.029703175582573488,
  "joint_prec": 0.05243567732106656,
  "joint_recall": 0.022821656831224828
}
```

句子粒度评估:

Evaluating prediction file: output/submissions/train_v1/
pred_seed_26_epoch_4_99999.json
Using gold file: ../data/dev.json

```
{
  "em": 0.23199208443271768,
  "f1": 0.3875065322194566,
  "prec": 0.45579313514834613,
  "recall": 0.38861707116580807,
  "sp_em": 0.7859498680738787,
  "sp_f1": 0.6051231310466144,
  "sp_prec": 0.6031662269129288,
  "sp_recall": 0.6090369393139842,
  "joint_em": 0.22348284960422163,
  "joint_f1": 0.32856448260556326,
  "joint_prec": 0.38527304924443834,
  "joint_recall": 0.3312264124700118
}
```

单独构建文档，做格式转换生成data文件夹，包括train和dev的json文件。然后直接运行代码，过程中对data_process训练cail数据做了调整：

调试过程总结:

- 转换数据格式时候， answer_start版本和readexample对

有 orig_answer_text 且成功对齐 answer_start, 可以训练 (标准抽取式 QA), 模型能学会 span 位置

有 orig_answer_text 但 answer_start_position == [], 无法训练

没有监督信号, span loss 无法计算

- 训练过程中大量样本因找不到答案开始位置而被丢弃;
- 修复尝试:
- 策略: 将 `answer = [-1]` (只做分类任务), 不

answerstartpostion和endpostion用句子span来对齐，修改了
answertext=对应的句子，解决了对齐由于把不同格式文件转换的过程
中无法复刻readexample原始样本一问一答结构的问题：

- 持续完善数据预处理逻辑;
- 探索 “无支

督时”的软标签机制。

監句持

[illegible]

改用粗粒度方式提取ans_postion，直接用sp的id查找已经分好的sentstart和end，赋值给ans_start_position,ans_end_position,交作业前结果是对齐的：

七、开发集评估

评估模块 (evaluate.py)

- 输入：
- --gold_file: 原始 dev.json
- --pred_file: 模型预测文件（包含 "answer" 和 "sp" 两部分）
- 输出：
- 控制台打印 JSON 格式的指标：例lawformer最后输出的

运行说明

训练后运行如下评估脚本：

字段名	含义	备注
em	精确匹配 (Exact Match)	预测答案与标准答案完全一致 (字符级)
f1	F1 分数	允许部分匹配, $F1 = 2PR / (P + R)$
prec	精确率 (Precision)	预测为正的中有多少是正确的
recall	召回率 (Recall)	标准答案中有多少被预测正确
sp_em	支持句精确匹配	预测支持句编号需与 gold 完全一致
sp_f1	支持句 F1 分数	支持句部分命中也能得分

字段名	含义	备注
sp_prec	支持句精确率	预测为支持句的中有多少是正确的
sp_recall	支持句召回率	gold 中的支持句被模型预测命中的比例
joint_em	联合精确匹配	答案和支持句都完全正确
joint_f1	联合 F1 分数	综合评估模型理解能力，考虑答案 + 支持句
joint_prec	联合精确率	同时预测正确的答案与支持句在预测中所占比例
joint_recall	联合召回率	同时正确预测的答案与支持句在 gold 中的覆盖比例

主要函数

- `f1_score(pred, gold)`: 基于字符的重叠计算答案的 F1、精确率、召回率。用于评估文字类答案（如 span）的 F1 值：
 - 将 prediction 和 gold 均标准化（通过 `normalize_answer`）；
 - 然后逐字符比较预测和 gold 之间的重叠程度（如“合同”与“合同纠纷”）；
 - 计算 精确率、召回率 与 F1 分数。
- `exact_match_score(pred, gold)`: 是否与 gold 完全一致。
- `update_answer()`: 更新 answer 部分指标。封装调用 `f1_score()` 与 `exact_match_score()`，并将结果写入指标字典 `metrics`，累计统计：
- `update_sp()`: 将预测的支持句与 gold 对比，统计 TP/FP/FN 以计算支持句指标。
 - prediction 是形如 `[["段落标题", 3], ["段落标题", 4]]` 的列表，代表模型预测第 3、4 句为支撑；
 - gold 是真实的支持句位置；
 - 两者转为 set 后交集求 TP、FP、FN，并计算：若 FP 和 FN 都为 0，说明预测和 gold 完全一致， $EM = 1$
- `evaluate()`: 读取预测和 gold 文件，循环统计每条数据的指标，计算总体平均值。

bash evaluate.sh

验证结果

```
{"em": 0.6587301587301587,
  "f1": 0.7336188227886133,
  "prec": 0.7554285661428518,
  "recall": 0.739656231835055,
  "sp_em": 0.376984126984127,
  "sp_f1": 0.5737969261778788,
  "sp_prec": 0.6407738095238096,
  "sp_recall": 0.5609126984126985,
  "joint_em": 0.29563492063492064,
  "joint_f1": 0.43933772027826007,
  "joint_prec": 0.5038246714337982,
  "joint_recall": 0.4368421406418255
}
```

简要分析如下：

 模型性能对比分析：

指标名称	roberta_chinese	lawformer (epoch 5)	lawformer (epoch 10)	cail_data (epoch 4) — +roberta
F1	0.7213	0.7286 ↑	0.7336 ↑	0.3875 ↓
EM	0.6468	0.6448 ↓	0.6587 ↑	0.2320 ↓
Precision	0.7450	0.7458 ≈	0.7554 ↑	0.4558 ↓
Recall	0.7248	0.7334 ↑	0.7397 ↑	0.3886 ↓

指标名称	roberta_chinese	lawformer (epoch 5)	lawformer (epoch 10)	cail_data (epoch 4)—+roberta
SP F1	0.6112	0.5504 ↓	0.5738 ↑ (仍低于RoBERTa)	0.6051 ≈
SP EM	0.4187	0.3631 ↓	0.3770 ↑	0.7859 ↑
SP Prec	0.6763	0.6423 ↓	0.6408 ↓	0.6032 ↓
SP Recall	0.5983	0.5239 ↓	0.5609 ↑	0.6090 ↑
Joint F1	0.4422	0.4170 ↓	0.4393 ≈	0.3286 ↓
Joint EM	0.3016	0.2837 ↓	0.2956 ↑	0.2235 ↓
Joint Prec	0.5085	0.4908 ↓	0.5038 ↓	0.3853 ↓
Joint Recall	0.4372	0.4029 ↓	0.4368 ≈	0.3312 ↓

- 整体准确性 (F1/EM) : lawformer (epoch 10) 表现最佳，略高于 roberta_chinese。

- 支持句指标 (SP F1) : lawformer 表现略逊色，但 epoch 10 相较 epoch 5 有所提升，cail_data 数据没有训练完，一半的数据已经超过了其他方案，所以增加数据是提升效果非常好的办法。

- 联合指标 (Joint F1) : roberta_chinese 仍略胜一筹，说明它在“答案+支持句”双任务联合表现更稳健。

最后一个输出：

```
output/submissions/train_v1/
pred_seed_51_epoch_10_99999.json
Using gold file: ../data/dev.json
```

附件：

Evaluate.py

pred_seed_13_epoch_10_99999.json—bert结果

pred_seed_68_epoch_5_99999.json—lawtrans结果 模型很大，硬盘爆掉断点重训到10epoch

pred_seed_24_epoch_7_3933.json-token级粒度cail预测结果

pred_seed_26_epoch_4_99999.json-句子级力度cail预测结果-由于时间来不及，还在训练，那最新的数据记录

train.sh, run_cail.py

preprocess.sh

evaluat.sh

data_process.py-cail数据修改

cailtomy.ipynb-尝试处理cail数据文件

附图：run_cail
输出结果：
output文件夹

/ ... / 法律智能问答案例 / baseline / ... / submissions / train_v1 /			
名称	名称	已修改	
output	pred_seed_13_epoch_10...	4天前	t A A A A A A A A A A
output_bermodel	pred_seed_13_epoch_10...	4天前	
data_cail	pred_seed_13_epoch_10...	4天前	
model	pred_seed_13_epoch_10...	4天前	
tools	pred_seed_13_epoch_10...	4天前	
lawformer	pred_seed_13_epoch_10...	4天前	
output_lawtransformer	pred_seed_13_epoch_10...	4天前	
processed	昨天		
processed_bermodel	4天前		

附图dataprocess输出： process文件夹

/ ... / baseline / processed_bermodel /		
名称		已修改
dev_feature.pkl.gz		4天前
dev_example.pkl.gz		4天前
train_feature.pkl.gz		4天前
train_example.pkl.gz		4天前